Online Training Program
ASP.NET Core Web API Online Training

ASP.NET WEB API – Basics
Introduction to ASP.NET Web API
Creating ASP.NET Web API Application
How to add Swagger in Web API
How to use Fiddler to test Web API
How to use POSTMAN to test Web API
ASP.NET Web API using SQL Server
Content Negotiation in Web API
Media Type Formatter in Web API
How to Implement GET Method in Web API
How to Implement POST Method in Web API
How to Implement PUT Method in Web API
How to Implement DELETE Method in Web API
Custom Method Names in Web API
Parameter Binding in Web API
Consuming Web API Service From jQuery
Calling Web API Service in a Cross Domain Using jQuery AJAX
Cross-Origin Resource Sharing in WEB API

WEB API Routing and Attribute Routing
ASP.NET Web API Routing
Routing Variations in WEB API
Routing and Action Selection in Web API
ASP.NET Web API Attribute Routing
Optional Parameters in Web API Attribute Routing
Route Prefix in Web API
Web API Attribute Routing Route Constraints
Route Names and Route Orders in Attribute Routing

ASP.NET WEB API – Security
Authentication and Authorization in ASP.NET Web API
ASP.NET Web API Basic Authentication
Role-Based Basic Authentication in Web API
Web API Service with Basic Authentication
HTTP Message Handlers in WEB API
Basic Authentication Using Message Handler in Web API
HTTP Client Message Handler in Web API
Token Based Authentication in ASP.NET Web API
Client Validation Using Basic Authentication in Web API
Refresh Token in Web API
Consume Refresh Token in C# Client
HMAC Authentication in Web API

Web API Versioning
Web API Versioning Using URI
Web API Versioning using Query string Parameter
Web API Versioning Using Custom Header
Web API Versioning Using Accept Header
Web API Versioning Using Custom Media Types

Advanced Concepts
Enable SSL in Visual Studio Development Server
Enable HTTPS in Web API

Popular ASP.NET Web API Books
Most Recommended ASP.NET Web API Books

Web API Attribute Routing Route Constraints

Back to: ASP.NET Web API Tutorials For Beginners and Professionals

ASP.NET Web API Attribute Routing Route Constraints

In this article, I will discuss the **Web API Attribute Routing Route Constraints** with examples. We are going to work with the same example that we worked in our previous articles. So, please read the following articles before proceeding to this article.

Attribute Routing in Web API

Optional URI Parameters and Default values in Attribute Routing

Attribute Routing Route Prefix in WEB API

Web API Attribute Routing Route Constraints

The Web API Attribute Routing Route Constraints are nothing but a set of rules that we can apply on our routing parameters to restrict how the parameters in the route template are matched. The general syntax is

{parameter:constraint}

Let us understand ASP.NET Web API Attribute Routing Route Constraints with one example.

Let's modify the Students Controller as shown below

```
namespace AttributeRoutingRouteConstraints
{
    [RoutePrefix("students")]
    public class StudentController : ApiController
    {
        static List<Student> students = new List<Student>();
        {
            new Student() { Id = 1, Name = "Pranaya" },
            new Student() { Id = 2, Name = "Priyanka" },
            new Student() { Id = 3, Name = "Anurag" },
            new Student() { Id = 4, Name = "Sambit" }
        };

        [HttpGet]
        [Route("{studentID}")]
        public Student GetStudentDetails(int studentID)
        {
            Student studentDetails = students.FirstOrDefault(s => s.Id == studentID);
            return studentDetails;
        }
    }
}
```

Now, if we navigate to **/students/1** URI, then the **GetStudentDetails(int studentID)** action is executed and we get the details of the student whose id is 1 as expected.

Let's change our business requirement, in addition to retrieving the student details by **'studentId'**, we also want to retrieve the student details by **'student Name'**. So let's add another GetStudentDetails() action method with a string parameter as shown below.

```
namespace AttributeRoutingRouteConstraints
{
    [RoutePrefix("students")]
    public class StudentController : ApiController
    {
        static List<Student> students = new List<Student>();
        {
            new Student() { Id = 1, Name = "Pranaya" },
            new Student() { Id = 2, Name = "Priyanka" },
            new Student() { Id = 3, Name = "Anurag" },
            new Student() { Id = 4, Name = "Sambit" }
        };

        [HttpGet]
        [Route("{studentID}")]
        public Student GetStudentDetails(int studentID)
        {
            Student studentDetails = students.FirstOrDefault(s => s.Id == studentID);
            return studentDetails;
        }

        [HttpGet]
        [Route("{studentName}")]
        public Student GetStudentDetails(string studentName)
        {
            Student studentDetails = students.FirstOrDefault(s => s.Name ==
studentName);
            return studentDetails;
        }
    }
}
```

At this point build the solution, and navigate to the following URI's

/students/1

/students/Pranaya

In both the cases we will get the below error:

Multiple actions were found that match the request: GetStudentDetails on type AttributeRoutingWEBAPI.Controllers.StudentsController GetStudentDetails on type AttributeRoutingWEBAPI.Controllers.StudentsController

This is because the WEB API Framework does not know or does not identify which version of the GetStudentDetails() action method to use. This is the situation where the route constraints play a very important role.

If an integer is specified in the URI like **/students/1**, then we need to execute the GetStudentDetails(int studentId) action method which takes an integer parameter whereas if a string is specified in the URI like **/students/Pranaya**, then we need to execute the GetStudentDetails(string studentName) action method which takes the parameter of type string.

This can be very easily achieved using Attribute Route Constraints in the WEB API application. To specify the attribute route constraint, the syntax is **{parameter:constraint}**. With these constraints in place, if the parameter segment in the URI is an integer, then the GetStudentDetails(int studentId) action method with integer parameter is invoked and if it is a string value then the GetStudentDetails(string studentName) action method with string parameter is invoked.

Let's modify the Student Controller to use the Attribute Route Constraints as shown below to achieve the above requirements.

```
namespace AttributeRoutingRouteConstraints
{
    [RoutePrefix("students")]
    public class StudentController : ApiController
    {
        static List<Student> students = new List<Student>();
        {
            new Student() { Id = 1, Name = "Pranaya" },
            new Student() { Id = 2, Name = "Priyanka" },
            new Student() { Id = 3, Name = "Anurag" },
            new Student() { Id = 4, Name = "Sambit" }
        };

        [HttpGet]
        [Route("{studentID:int}")]
        public Student GetStudentDetails(int studentID)
        {
            Student studentDetails = students.FirstOrDefault(s => s.Id == studentID);
            return studentDetails;
        }

        [HttpGet]
        [Route("{studentName:alpha}")]
        public Student GetStudentDetails(string studentName)
        {
            Student studentDetails = students.FirstOrDefault(s => s.Name ==
studentName);
            return studentDetails;
        }
    }
}
```

Now build the solution, and navigate to the following two URIs and see everything is working as expected.

/students/1

/students/Pranaya

Please note that **'alpha'** stands for uppercase or lowercase alphabet characters. Along with alpha and int, you can also use constraints such as decimal, float, long, double, bool, etc. Please check the following MSDN link for the full list of available constraints in web API.

https://docs.microsoft.com/en-us/aspnet/web-api/overview/web-api-routing-and-actions/attribute-routing-in-web-api-2#route-constraints

Example:

If you want GetStudentDetails(int studentId) action method to be mapped to URI **/students/{studentId}**, only if the studentId is a number greater than ZERO, then use the 'min' constraint as shown below.

```
[HttpGet]
[Route("{studentID:int:min(1)}")]
public Student GetStudentDetails(int studentID)
{
    Student studentDetails = students.FirstOrDefault(s => s.Id == studentID);
    return studentDetails;
}
```

With the above change, if we specify a positive number like 1 in the URI, then it will be mapped to the GetStudentDetails(int studentID) action method as expected

/students/1

However, if we specify 0 or a negative number less than ZERO, then we will get an error. For example, if we specify 0 as the value for studentID in the URI,

/students/0

We will get the below error

localhost:50470/students/0

HTTP Error 404.0 - Not Found

The resource you are looking for has been removed, had its name changed, or is temporarily unavailable.

Along with the 'min' constraint, you can also specify the 'max' constraint as shown below. For example, if you want the studentID value in the URI to be between 1 and 3 inclusive, then you can specify both 'min' and 'max' constraints as shown below.

```
[HttpGet]
[Route("{studentID:int:min(1):max(3)}")]
public Student GetStudentDetails(int studentID)
{
    Student studentDetails = students.FirstOrDefault(s => s.Id == studentID);
    return studentDetails;
}
```

The above example can also be achieved using the "range" attribute as shown below

```
[HttpGet]
[Route("{studentID:int:range(1,3)}")]
public Student GetStudentDetails(int studentID)
{
    Student studentDetails = students.FirstOrDefault(s => s.Id == studentID);
    return studentDetails;
}
```

Custom Web API Route Constraints in Attribute Routing

You can also create custom route constraints in Web API and to do so you need to implement the **IHttpRouteConstraint** interface. For example, the below constraint will restrict a parameter value to a non-zero integer value.

```
using System.Collections.Generic;
using System.Globalization;
using System.Net.Http;
using System.Web.Http.Routing;

namespace AttributeRoutingInWEBAPI.Models

    public class NonZeroConstraint : IHttpRouteConstraint
    {
        public bool Match(HttpRequestMessage request, IHttpRoute route, string
parameterName,
            IDictionary<string, object> values, HttpRouteDirection routeDirection)
        {
            object value;
            if (values.TryGetValue(parameterName, out value) && value != null)
            {
                long longValue;
                if (value is long)
                {
                    longValue = (long)value;
                    return longValue != 0;
                }

                string valueString = Convert.ToString(value,
CultureInfo.Invariant);
                if (Int64.TryParse(valueString, NumberStyles.Integer,
                    CultureInfo.Invariant, out longValue))
                {
                    return longValue != 0;
                }
                else
                {
                    return false;
                }
            }
            return false;
        }
    }
```

The following code shows how to register the custom constraint:

```
namespace AttributeRoutingInWEBAPI.App_Start
{
    public static class WebApiConfig
    {
        public static void Register(HttpConfiguration config)
        {
            var constraintResolver = new DefaultInlineConstraintResolver();
            constraintResolver.ConstraintMap.Add("nonzero", typeof(NonZeroConstraint));

            // Attribute routing.
            config.MapHttpAttributeRoutes((constraintResolver);

            // Convention-based routing.
            config.Routes.MapHttpRoute(
                name: "DefaultApi",
                routeTemplate: "api/{controller}/{id}",
                defaults: new { id = RouteParameter.Optional }
                );
        }
    }
}
```

Now you can apply the custom constraint in your routes as shown below.

```
[HttpGet]
[Route("{studentName:alpha}")]
public Student GetStudentDetails(string studentName)
{
    Student studentDetails = students.FirstOrDefault(s => s.Name == studentName);
    return studentDetails;
}
```

In the next article, I am going to discuss **Route Names and Route Orders in WEB API**. Here, in this article, I try to explain the **Web API Attribute Routing Route Constraints** with examples. I hope this **Web API Attribute Routing Route Constraints** article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.

Dot Net Tutorials
About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

4 thoughts on "Web API Attribute Routing Route Constraints"

MARIO
NOVEMBER 8, 2020 AT 7:23 PM

You are not applying the custom route constraint in the last code snippet. Type I suppose

Reply

MARIO
NOVEMBER 8, 2020 AT 7:26 PM

I guess the route attribute should be {studentName:nonzero}

Reply

AJAY
FEBRUARY 3, 2020 AT 5:39 PM

It is not work outing...

Reply

ABD AKHTAR
APRIL 5, 2020 AT 9:34 PM

This should be
[HttpGet]
//[Route("{studentID:int:min(1):max(3)}/details")]
[Route("{studentID:int:nonzero}/details")]
public Student GetStudentDetails(int studentID)
{
Student studentDetails = students.FirstOrDefault(s => s.Id == studentID);
return studentDetails;
}

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name* Email* Website

Post Comment

© Dot Net Tutorials | Website Design by Sunrise Pixel